

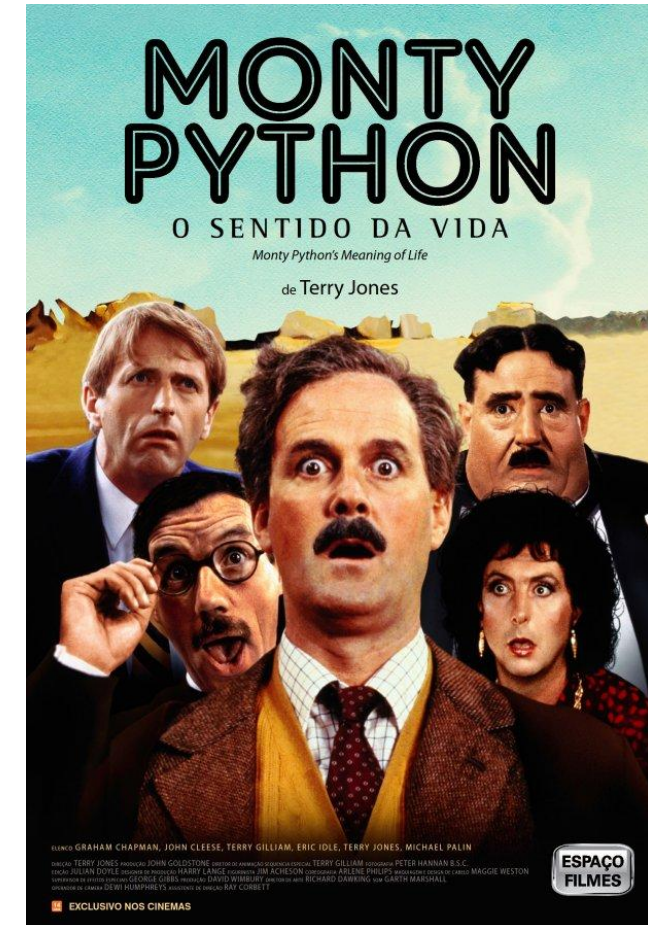
Introdução a linguagem Python



Pythonidae (Python family)



?



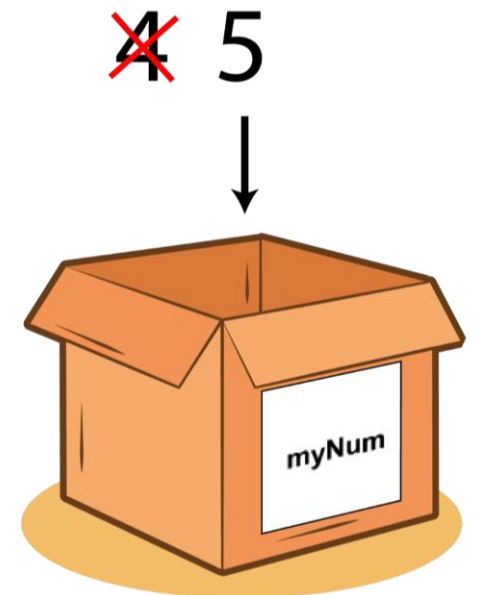
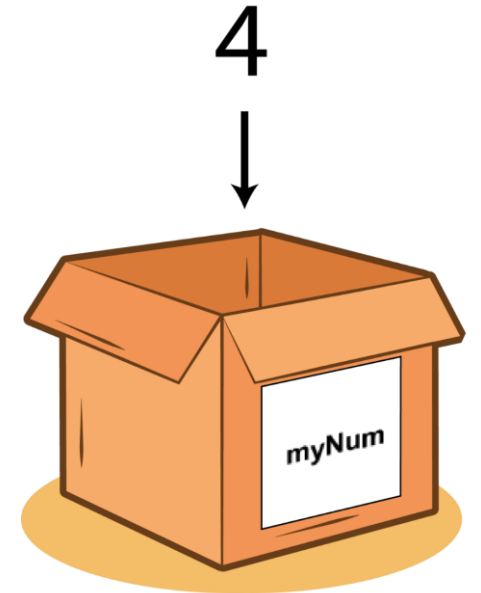
Professor: Alex Pereira

Variáveis

- Label (etiqueta) / Nome e Conteúdo
 - O programador escolhe o label e o conteúdo
 - ✓ Para satisfazer um requisito ou objetivo

```
myNum = 4  
print(myNum)  
myNum = 5  
print(myNum)
```

```
4  
5
```



Sintaxe do Python

- Dois pontos sinaliza o início de uma sentença composta
 - O conteúdo das sentenças compostas é **aninhado com espaços/tabs**
 - ✓ Sem chaves
- Ponto e virgula para finalizar uma sentença é opcional

Erro Comum
Esquecer de aninhar
sentenças compostas

```
x = 2  # Atribuição com sinal de = (igual)
if x > 0:
    # adiciona 1 a x
    x = x + 1;
    print(x)
    print('x maior que zero')
else:
    print('x menor ou igual a zero')
```

```
3
x maior que zero
```

Sintaxe do Python

- Referências para objetos não possuem um tipo associado
 - Não há problema em reatribuir uma referência a outro tipo
 - ✓ Por exemplo:

```
x = 2
print(type(x))
x = 'carro'
print(type(x))
```

```
<class 'int'>
<class 'str'>
```

Sintaxe do Python

- Funções são declaradas com a palavra-chave **def**
 - Qual a utilidade das funções ?
 - ✓ Reusabilidade / legibilidade
 - Uma função precisa ser carregada em memória
 - ✓ para ser encontrada pelo interpretador
- Chamada / execução de uma função
 - `nome_funcao()`

Erro Comum

Esquecer de aninhar
sentenças compostas

```
def imprimir(a):
```

```
    print(a)
```

```
imprimir(2)
```

2

return de uma função Python

- Explícito

- com o keyword **return**

- ✓ seguido de um valor

- Exemplo: `return 10`

```
def soma(m, n):  
    ←→ return m + n  
result = soma(4, 5)  
print(result)
```

9

- Implícito

- sem o keyword **return**, o retorno é **None**

- ✓ vide slide anterior

- Erro comum!

- Precisar de um retorno mas esquecer de escrevê-lo

Revisão: Definição e execução de uma função

```
def soma(a, b):  
    s = a + b  
    return s
```

The diagram illustrates the definition of a function `soma`. It features a yellow background with a vertical blue line on the left. Numbered blue circles (1-6) and red arrows indicate the flow of execution. Circle 1 is above `def`, circle 2 is above `soma`, circle 3 is above the opening parenthesis `(`, circle 4 is above the closing parenthesis `)`, circle 5 is to the left of the assignment line, and circle 6 is above `return`. A red arrow points from circle 5 to the assignment line, and another red arrow points from the `return` statement back to the left margin.

```
result = soma(1, 2)
```

The diagram illustrates the execution of the function `soma`. It features a yellow background. Numbered blue circles (7-11) indicate the flow of execution. Circle 7 is below `result`, circle 8 is below `=`, circle 9 is below `soma`, circle 10 is above the first argument `1`, and circle 11 is above the second argument `2`.

Dados do tipo Escalar

- Guardam valores únicos (single) ou simples
- None
 - A representação de valor nulo em Python
- float
 - Número em formato ponto flutuante de 64-bit (Exemplo: 3.1290)
- bool
 - Um valor True ou False
- int
 - Um número inteiro
- Tipo sequencial
 - str
 - ✓ Conjunto de caracteres. Guarda strings codificadas com UTF-8
 - bytes
 - ✓ bytes ASCII (ou bytes codificados como Unicode)

Operadores Binários

Operação	Descrição
a + b	Soma a e b
a - b	Subtrai a de b
a * b	Multiplica a por b
a / b	Divide a por b
a // b	Divisão inteira de a por b. Desconsidera o resto.
a ** b	Eleva a à potência b
a AND b	True se a e b são True
a OR b	True se a ou b são True

* Em negrito os mais importantes

Operadores Binários

Operação	Descrição
a == b	True se a é igual a b
a != b	True se a não é igual a b
a < b, a <= b	True se a é menor (ou menor ou igual) a b
a > b, a >= b	True se a é maior (ou maior ou igual) a b
a is b	True se a e b referenciam o mesmo objeto Python
a is not b	True se a e b referenciam objetos Python distintos

* Em negrito os mais importantes

Conversão de Tipos

- `str`, `bool`, `int`, e `float`
 - também são funções para converter valores para esses tipos

```
s = '3.14159'  
fval = float(s)  
ival = int(fval)  
print(fval)  
print(ival)
```

```
3.14159  
3
```

Laços (loops) do tipo While

- Uma maneira de iterar sobre uma coleção
 - **enquanto** uma condição for verdadeira

```
n = 0
total = 0
while n < 4:
    total = total + n*n
    n = n + 1

print(total)
```

Laços (loops) do tipo for

- Uma maneira de iterar sobre uma coleção
- Usa-se o keyword **in** para referenciar a coleção.
 - **val** foi um termo escolhido pelo programador
 - **break** interrompe a iteração

Atente para a correta
indentação

```
sequence = [1, 2, 0, 4, 6, 5, 2, 1]
tot_until_5 = 0
for val in sequence:
    if val == 5:
        break
    tot_until_5 = tot_until_5 + val
print(tot_until_5)
```

Laço do tipo for com a função range

- range(5)
 - retorna um iterator da sequência 0, 1, 2, 3, 4
- range(2, 6)
 - retorna um iterator da sequência 2, 3, 4, 5

```
for val in range(3):  
    print(val*val)
```

```
0  
1  
4
```

Formando uma string (texto)

- Strings podem ser definidas usando aspas simples ou duplas
 - 'aluno' ou "aluno" são válidos

```
a = 4.5
```

```
b = 2
```

```
frase1 = f"a={a}, b={b}"
```

```
frase2 = "a={0}, b={1}".format(a, b)
```

```
print(frase1)
```

```
print(frase2)
```

```
a=4.5, b=2
```

```
a=4.5, b=2
```

Funções Populares de String: join

- Transforma uma lista de strings numa string
 - concatenada por um separador

```
lista_str = ["1", "2", "3", "4"]  
str_concatenado = "-".join(lista_str)  
print(str_concatenado)
```

```
1-2-3-4
```


Funções Populares de String: split

- `str.split(sep=None, maxsplit=-1)`
 - Divide uma string em partes separadas por um caractere separador *sep*
 - ✓ o resultado retornado é uma lista dos sub-elementos

```
texto = "1-2-3-4"  
lista = texto.split("-")  
print(lista)
```

```
['1', '2', '3', '4']
```

Outras operações com strings

- Um character * numero

- Ex.: "*" * 10

```
"*" * 10
```

```
'*****'
```

- Concatenação de strings

```
"hello " + "world!"
```

```
'hello world!'
```

Estruturas de Dados do Python (Nativas)

list (Lista)

- Sequência de tamanho variável e conteúdo mutável (alterável)
 - Pode conter objetos de vários tipos
 - Define-se uma lista com colchetes []
 - `append` (inserir elementos), `pop` (remover elementos pelo índice)

```
al = [2, 4, 0, None]
```

```
print(al)
```

```
al.append(9)
```

```
print(al)
```

```
al.pop(1) # Remover pelo índice
```

```
print(al)
```

```
al.remove(0) # Use remove para remover pelo valor
```

```
print(al)
```

```
[2, 4, 0, None]
```

```
[2, 4, 0, None, 9]
```

```
[2, 0, None, 9]
```

```
[2, None, 9]
```

Combinando Listas

- Use o operador + ou a função extend
 - A função extend é mais rápida do que o operador +

```
al = [2, 4, 0, None]  
al_plus = al + ['a', 'b']  
al.extend(['a', 'b'])  
comp = al_plus == al  
print(comp)
```

True

Slicing (fatiar)

- Use intervalos entre colchetes para fatiar sequências

```
al = [2, 4, 0, 3, 7, 10, 4, 5]
print(al[0:3])    # Do índice zero até o 3 (não incluso)
print(al[:4])     # Do índice zero ao 4
print(al[2:])     # Do índice 2 até o último
print(al[-1])     # O último elemento
print(al[::2])    # A cada dois elementos a partir do zero
print(al[::-1])  # Reverter/espelhar os elementos
al[1:3] = [8, 8]
print(al)
```

Tamanho de uma lista: `len(lista)`

- o método **len** retorna o tamanho de uma lista

```
al = [2, 4, 0, 3, 7, 10, 4, 5]  
print(len(al))
```

```
8
```

Importar (*import*) módulos

- Módulo
 - arquivo de extensão .py
 - ✓ Contendo definições e funções/métodos
- Package (pacote)
 - diretório/pasta marcado com a presença de um arquivo `__init__.py`
 - ✓ Serve para armazenar módulos
 - Instalar um pacote: **pip install** <nome_pacote> (Ex.: **pip install numpy**)
 - No Google colab: **!pip install** <nome_pacote> (Ex.: **!pip install numpy**)

```
import os  
  
print(os.getcwd())
```

C:\Users\alex\Google Drive\Aulas\Enap...

```
from os import getcwd  
  
print(getcwd())
```

C:\Users\alex\Google Drive\Aulas\Enap...

Conceitos de Python abordados na aula

- def
- If
- else
- while
- for
- in
- break
- return
- import
- from

